

PREREQUISITE TASK

```
CREATE TABLE labs.person (  
  id integer NOT NULL,  
  name varchar(15)  
);  
  
INSERT INTO labs.person VALUES(1, 'Bob');  
INSERT INTO labs.person VALUES(2, 'Alice');  
INSERT INTO labs.person VALUES(3, 'Robert');
```

EXPLAIN ANALYZE

```
select * from labs.person
```

1	×	
IN ANALYZE select * from labs.person		
Enter a SQL expression to filter results (use Ctrl+Space)		
A-Z QUERY PLAN		
Append (cost=0.00..1.06 rows=5 width=18) (actual time=0.054..0.064 rows=4.00 loops=1)		
Buffers: shared hit=1 dirtied=1		
-> Seq Scan on person person_1 (cost=0.00..1.04 rows=4 width=18) (actual time=0.053..0.053 rows=4.00 loops=1)		
Buffers: shared hit=1 dirtied=1		
-> Seq Scan on users person_2 (cost=0.00..0.00 rows=1 width=17) (actual time=0.008..0.008 rows=0.00 loops=1)		
Planning:		
Buffers: shared hit=128 read=1		
Planning Time: 4.007 ms		
Execution Time: 0.131 ms		

TASK 1 - PERFORMANCE WITH UNLOGGED TABLES

A standard (logged) tables write every change to the WAL, and this takes more time. An UNLOGGED table skips WAL writes entirely, so inserts are faster.

```
insert into test_simple values (generate_series(1,1000000));
```

```
--task2
create table labs.test_simple(a int,b int)

EXPLAIN ANALYZE
insert into labs.test_simple values (generate_series(1,1000000));
```

Results 1 X

EXPLAIN ANALYZE insert into labs.test_simple values (Enter a SQL expression to filter results (use Ctrl+Space)

A-Z QUERY PLAN

1	Insert on test_simple (cost=0.00..5000.02 rows=0 width=0) (actual time=1346.123..1346.124 rows=0.00 loops=1)
2	Buffers: shared hit=1008845 dirtied=4425 written=4429
3	-> ProjectSet (cost=0.00..5000.02 rows=1000000 width=8) (actual time=0.004..64.921 rows=1000000.00 loops=1)
4	-> Result (cost=0.00..0.01 rows=1 width=0) (actual time=0.001..0.002 rows=1.00 loops=1)
5	Planning Time: 0.038 ms
6	Execution Time: 1346.148 ms

insert into test_simple values (generate_series(1,5000000));

```
--task2
create table labs.test_simple(a int,b int)

EXPLAIN ANALYZE
insert into labs.test_simple values (generate_series(1,1000000));

EXPLAIN ANALYZE
insert into labs.test_simple values (generate_series(1,5000000));
```

Results 1 X

EXPLAIN ANALYZE insert into labs.test_simple values (Enter a SQL expression to filter results (use Ctrl+Space)

A-Z QUERY PLAN

1	Insert on test_simple (cost=0.00..25000.02 rows=0 width=0) (actual time=6597.366..6597.366 rows=0.00 loops=1)
2	Buffers: shared hit=5044254 dirtied=22124 written=29322
3	-> ProjectSet (cost=0.00..25000.02 rows=5000000 width=8) (actual time=0.003..307.213 rows=5000000.00 loops=1)
4	-> Result (cost=0.00..0.01 rows=1 width=0) (actual time=0.001..0.001 rows=1.00 loops=1)
5	Planning Time: 0.022 ms
6	Execution Time: 6597.385 ms

insert into test_unlogged values (generate_series(1,1000000));

```
EXPLAIN ANALYZE
INSERT INTO labs.test_unlogged VALUES (generate_series(1,1000000));
```

Results 1 X

EXPLAIN ANALYZE INSERT INTO labs.test_unlogged VA Enter a SQL expression to filter results (use Ctrl+Space)

A-Z QUERY PLAN

1	Insert on test_unlogged (cost=0.00..5000.02 rows=0 width=0) (actual time=558.916..558.917 rows=0.00 loops=1)
2	Buffers: shared hit=1008845 dirtied=4425 written=4429
3	-> ProjectSet (cost=0.00..5000.02 rows=1000000 width=8) (actual time=0.003..68.807 rows=1000000.00 loops=1)
4	-> Result (cost=0.00..0.01 rows=1 width=0) (actual time=0.001..0.002 rows=1.00 loops=1)
5	Planning Time: 0.039 ms
6	Execution Time: 558.943 ms

insert into test_unlogged values (generate_series(1,5000000));

EXPLAIN ANALYZE
INSERT INTO labs.test_unlogged VALUES (generate_series(1,5000000));

Results 1

PLAIN ANALYZE INSERT INTO labs.test_unlogged VA

AZ QUERY PLAN

Insert on test_unlogged (cost=0.00..25000.02 rows=0 width=0) (actual time=3312.372..3312.373 rows=0.00 loops=1)
Buffers: shared hit=5044254 dirtied=22124 written=41938
-> ProjectSet (cost=0.00..25000.02 rows=5000000 width=8) (actual time=0.004..381.750 rows=5000000.00 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=0) (actual time=0.001..0.001 rows=1.00 loops=1)
Planning Time: 0.023 ms
Execution Time: 3312.391 ms

TASK 2– UNDERSTANDING INHERITED TABLES

Table creation and initial inserts were made

SELECT * FROM labs.users;

123 id	A-Z name	123 user_id	A-Z login
1	new Bob	1	NewLogin
999	TestUser	2	TestLogin

SELECT * FROM labs.person;

123 id	A-Z name
1	Bob
2	Alice
3	Robert
1	new Bob
999	TestUser

SELECT * FROM ONLY labs.person;

123 id	A-Z name
1	Bob
2	Alice
3	Robert

UPDATE person query results in duplicates

123 id	A-Z name
2	Alice
3	Robert
1	not Bob
999	TestUser
1	not Bob

UPDATE person ONLY query changes the primary person table Bob name

123 id	A-Z name
2	Alice
3	Robert
1	Bob on update
999	TestUser
1	not Bob

ALTER TABLE labs.person **ADD COLUMN** status integer **DEFAULT** 0;

Users table also inherits the column

123 id	A-Z name	123 user_id	A-Z login	123 status
999	TestUser	2	TestLogin	0
1	not Bob	1	NewLogin	0

ALTER TABLE labs.person **ADD CONSTRAINT** status **CHECK** (status in (0,1)) **NO INHERIT**;

Inserting the value with status 2 into labs.person fails

SQL Error [23514]: ERROR: new row for relation "person" violates check constraint "status" Detail: Failing row contains (100, Test, 2). Error position:	INSERT INTO labs.person (id, name, status) VALUES (100, 'Test', 2)
---	--

Inserting the value with status 2 into labs.user is possible (no inherit)

	INSERT INTO labs.person (id, name, status) VALUES (100, 'Test', 2);
	INSERT INTO labs.users (id, name, status, login) VALUES (101, 'Test2', 2, 'login2')

users 1	person 2	person 3	Statistics 4	person 5	Statistics 6	person 7
Name	Value					
Updated Rows	1					
Execute time	0.002s					
Start time	Thu Jul 09 23:42:53 EEST 2026					
Finish time	Thu Jul 09 23:42:53 EEST 2026					
Query	INSERT INTO labs.users (id, name, status, login) VALUES (101, 'Test2', 2, 'login2')					

ALTER TABLE labs.person **ADD CONSTRAINT** id **UNIQUE** (id, name);

The constraint works on person table only, the users table may bypass it (no inherit)

123 id	A-Z name	123 status
2	Alice	0
3	Robert	0
1	Bob on update	0
1	not Bob again	0
999	TestUser	0
1	not Bob	0
101	Test2	2
1	not Bob again	0

TASK 3 - BRIN VS B-TREE

B-TREE index execution time

Results 1	Statistics 2
Name	Value
Updated Rows	0
Execute time	7.827s
Start time	Fri Jul 31 22:47:04 EEST 2026
Finish time	Fri Jul 31 22:47:12 EEST 2026
Query	CREATE INDEX idx_test_index_btree ON labs.test_index USING btree (load_date)

BRIN index execution time

Results 1	Statistics 2
Name	Value
Updated Rows	0
Execute time	1.089s
Start time	Fri Jul 31 22:49:21 EEST 2026
Finish time	Fri Jul 31 22:49:22 EEST 2026
Query	CREATE INDEX idx_test_index_brin ON labs.test_index USING brin (load_date)

The created table test_index size:

A-Z pg_size_pretty
666 MB

The execution time of the query without indexes:

27	Execution Time: 2532.332 ms
----	-----------------------------

The execution time of the query with B-Tree index:

15	Execution Time: 295.408 ms
----	----------------------------

And its size:

A-Z pg_size_pretty
1 338 MB

The execution time of the query with BRIN index:

27	Execution Time: 299.664 ms
----	----------------------------

And its size:

A-Z pg_size_pretty
1 40 kB

Conclusion:

BRIN and B-Tree gave nearly identical query speedup, but BRIN's index size was significantly smaller (40 kB vs 338 MB).

TASK 4 - GIN VS GIST

GIST B- index execution time

Name	Value
Updated Rows	0
Execute time	4m 24s
Start time	Fri Jul 31 22:53:23 EEST 2026
Finish time	Fri Jul 31 22:57:47 EEST 2026
Query	CREATE INDEX idx_test_index_gist ON labs.test_index USING gist (t_hash gist_trgm_ops)

GIN index execution time

Name	Value
Updated Rows	0
Execute time	1m 21s
Start time	Fri Jul 31 23:00:44 EEST 2026
Finish time	Fri Jul 31 23:02:06 EEST 2026
Query	CREATE INDEX idx_test_index_gin ON labs.test_index USING gin (t_hash gin_trgm_ops)

The size of the created table is 652 MB

GIST Index query time:

12	Execution Time: 1061.680 ms
----	-----------------------------

Gist index size: 1769MB

3. GIN Index query time:

12	Execution Time: 14.950 ms
----	---------------------------

Gin index size: 600MB

Conclusion:

GIN wins on both parameters: it's significantly faster than GiST (14.95 ms vs 1061.68 ms) *and* smaller (600 MB vs 1769 MB, GiST is actually bigger than the table itself).

TASK 5 - CSV FILE AS A TABLE

```
CREATE EXTENSION IF NOT EXISTS file_fdw;
CREATE SERVER test_import FOREIGN DATA WRAPPER file_fdw;

CREATE FOREIGN TABLE labs.test_foreign_table
(
  LatD INT,
  LatM INT,
  LatS INT,
  NS TEXT,
  LonD INT,
  LonM INT,
  LonS INT,
  EW TEXT,
  City TEXT,
  State TEXT
)
SERVER test_import
OPTIONS
(
  filename 'C:\datasets\cities_list.csv',
  format 'csv',
  header 'true',
  delimiter ','
);

SELECT * FROM labs.test_foreign_table;
SELECT count(*) FROM labs.test_foreign_table;
```

test_foreign_table 1 × test_foreign_table 1 (2)

SELECT * FROM labs.test_foreign_table; SELECT count(*) FROM labs.test_forei Data filter is not supported

	latd	latm	lats	ns	lond	lonm	lons	ew	city	state
1	41	5	59	N	80	39	0	W	Youngstown	OH
2	42	52	48	N	97	23	23	W	Yankton	SD
3	46	35	59	N	120	30	36	W	Yakima	WA
4	42	16	12	N	71	48	0	W	Worcester	MA
5	43	37	48	N	89	46	11	W	WisconsinDells	WI
6	36	5	59	N	80	15	0	W	Winston-Salem	NC
7	49	52	48	N	97	9	0	W	Winnipeg	MB
8	39	11	23	N	78	9	36	W	Winchester	VA
9	34	14	24	N	77	55	11	W	Wilmington	NC
10	39	45	0	N	75	33	0	W	Wilmington	DE

The materialized view on foreign table was created

Statistics 1	Statistics 2	test_foreign_table 3	test_forei
Name	Value		
Updated Rows	128		
Execute time	0.03s		
Start time	Fri Jul 10 00:45:22 EEST 2026		
Finish time	Fri Jul 10 00:45:22 EEST 2026		
Query	CREATE MATERIALIZED VIEW labs.mview_test_foreign AS SELECT * FROM labs.test_foreign_table		

The row count is 128


```
SELECT count(*) FROM labs.test_foreign_table;
```

foreign_table 1

CT count(*) FROM labs.test_foreign_table | Enter a SQL expression to filter result

123 count
128

After the deleting rows on csv:

```
SELECT count(*) FROM labs.test_foreign_table;
```

Rows count 114

```
SELECT count(*) FROM labs.test_foreign_table;
```

test_foreign_table 1

SELECT count(*) FROM labs.test_foreign_table | Enter a SQL expression to

123 count
1 114

```
SELECT count(*) FROM labs.mview_test_foreign;
```

Rows count 128

```
SELECT count(*) FROM labs.mview_test_foreign;
```

w_test_foreign 1

CT count(*) FROM labs.mview_test_foreign | Enter a SQL expression to

123 count
128

After the refreshing materialized view:

```
SELECT count(*) FROM labs.mview_test_foreign;
```

Rows count 114

mview_test_foreign 1

SELECT count(*) FROM labs.mview_test_foreign | Enter a SQL expression to filter results (use Ctrl+Space)

123 count
1 114

Conclusion:

A foreign table created has no stored data of its own — every query re-reads the file, so the row count updated to 114 immediately after the CSV was edited. A materialized view, by contrast, physically stores a snapshot of its query result at creation time — it kept showing 128 even after the file changed, because it wasn't re-querying the source. Only after refresh it re-execute its query and updates to 114..